

Modélisation du Web et algorithme PageRank

Rémi PARROT, Joachim ROBERT, Ryan WONG-TZE-KIOON

11 mai 2026

Table des matières

1	Introduction	2
2	Modélisation du Web	2
2.1	Représentation du Web par un graphe orienté	2
2.2	Matrice d'adjacence	2
2.3	Matrice de transition	3
2.4	Limites du modèle	3
3	La matrice de Google	3
3.1	Principe de la téléportation	3
3.2	Définition de la matrice de Google	4
3.3	Propriétés de la matrice G	4
3.4	Interprétation probabiliste	4
4	Modélisation par une chaîne de Markov	4
4.1	Vecteur de probabilité	4
4.2	Évolution du système	5
4.3	Recherche d'un état stationnaire	5
4.4	Interprétation	5
5	Existence et unicité du PageRank	5
5.1	Existence et unicité d'un point fixe	5
5.2	Conséquence pour le PageRank	6
6	Implémentation en Python	7
6.1	Structure du programme	7
6.2	Génération du graphe	7
6.3	Construction de la matrice de transition	7
6.4	Construction de la matrice de Google	8
6.5	Calcul du PageRank	8
7	Premiers résultats	9
7.1	Graphe généré et scores PageRank	9
7.2	Convergence de l'algorithme	10
8	Réalisme et Optimisations	10
8.1	Utilisation de matrices creuses	10
8.2	Exemple sur un graphe creux	12
8.3	Influence du paramètre α	13
9	Conclusion	15

1 Introduction

Avec l'essor du Web, les moteurs de recherche sont devenus des outils incontournables pour naviguer dans une masse d'informations qui ne cesse de croître. Avant que Google ne s'impose comme moteur de recherche dominant, c'est Yahoo qui était privilégié pour effectuer des recherches sur Internet. À l'époque, les différents sites étaient regroupés autour d'un sujet commun, et ces regroupements étaient gérés entièrement par des humains. Par ailleurs, les temps de chargement des pages étaient bien plus longs qu'aujourd'hui, et Yahoo avait fait le choix de concentrer sur sa page d'accueil tout ce qu'un utilisateur serait susceptible de vouloir : actualités, météo, messagerie, etc.

Google a pris le contre-pied de cette approche sur deux points. D'abord, sa page de recherche se résumait à une simple barre de requête et un logo, ce qui la rendait nettement plus rapide à charger. Ensuite, et surtout, Google a privilégié une méthode automatique pour classer les pages selon un critère de pertinence, sans recourir à une organisation humaine. C'est précisément ce que propose l'algorithme *PageRank*, développé par Sergey Brin et Larry Page à la fin des années 1990 : un moyen d'ordonner les pages non seulement par pertinence thématique, mais aussi par importance relative, de façon entièrement automatisée. Couplé à une interface minimaliste, cet algorithme a permis à Google de prendre rapidement l'ascendant sur le marché pour devenir l'incontournable qu'il est aujourd'hui.

Le principe de base du *PageRank* est assez intuitif : une page Web est d'autant plus importante qu'elle est citée par d'autres pages, et surtout par des pages elles-mêmes bien référencées. Cette logique circulaire conduit naturellement à modéliser le Web comme un graphe orienté, où chaque page correspond à un sommet et chaque lien hypertexte à une arête orientée. À partir de ce modèle, on peut voir la navigation d'un utilisateur comme celle d'un *surfeur aléatoire* qui se déplace de page en page en cliquant sur des liens au hasard. Son comportement suit alors une dynamique probabiliste que l'on formalise à l'aide d'une chaîne de Markov. Le classement des pages revient alors à trouver la distribution stationnaire de cette chaîne, c'est-à-dire la fréquence limite avec laquelle chaque page est visitée.

Ce projet a pour but de détailler cette modélisation et l'algorithme qui en découle. On commencera par construire le modèle de graphe orienté et la matrice de transition associée au surfeur aléatoire. On introduira ensuite la matrice de Google, qui corrige certaines limitations du modèle de base, notamment les puits et les composantes déconnectées, et garantit l'existence d'une distribution stationnaire unique. Pour finir, on implémentera le calcul du PageRank en Python et on observera son comportement sur des graphes générés aléatoirement.

2 Modélisation du Web

2.1 Représentation du Web par un graphe orienté

On modélise l'ensemble des pages du Web par un graphe orienté $G = (V, E)$.

- Chaque sommet représente une page web.
- Chaque arête orientée représente un lien hypertexte.

Si une page j contient un lien vers une page i , on note $j \rightarrow i$.

Cette modélisation permet d'utiliser des outils d'algèbre linéaire et de théorie des graphes pour étudier la structure du Web.

2.2 Matrice d'adjacence

Soit n le nombre de pages.

On définit la matrice d'adjacence $A \in \{0, 1\}^{n \times n}$ par :

$$a_{ij} = \begin{cases} 1 & \text{si la page } j \text{ pointe vers la page } i, \\ 0 & \text{sinon.} \end{cases}$$

Ainsi, la colonne j de la matrice A représente l'ensemble des liens sortants de la page j .

2.3 Matrice de transition

On suppose maintenant qu'un internaute parcourt le Web de manière aléatoire : à chaque étape, il choisit uniformément un des liens sortants de la page courante.

On définit alors la matrice de transition P par :

$$P_{ij} = \begin{cases} \frac{1}{\deg(j)} & \text{si } j \rightarrow i, \\ 0 & \text{sinon,} \end{cases}$$

où $\deg(j)$ désigne le degré sortant de la page j , c'est-à-dire :

$$\deg(j) = \sum_{i=1}^n a_{ij}.$$

La matrice P vérifie la propriété suivante :

$$\sum_{i=1}^n P_{ij} = 1$$

pour toute colonne j telle que $\deg(j) \neq 0$.

Ainsi, P est une matrice de vecteurs colonnes stochastiques ou nuls.

2.4 Limites du modèle

Ce modèle présente cependant deux difficultés principales :

- Certaines pages peuvent ne posséder aucun lien sortant.
- Le graphe peut être composé de plusieurs composantes non connectées, empêchant l'existence d'une distribution stationnaire unique.

Ces problèmes seront résolus dans la section suivante grâce à l'introduction de la matrice de Google.

3 La matrice de Google

3.1 Principe de la téléportation

Le modèle précédent, basé uniquement sur la matrice de transition P , peut poser des problèmes d'existence et d'unicité de la distribution stationnaire. Pour y remédier, on introduit un mécanisme dit de téléportation. À chaque étape, le surfeur a deux comportements possibles :

- suit un lien sortant avec une probabilité α ,
- se téléporte vers une page choisie uniformément au hasard avec une probabilité $1 - \alpha$.

Le paramètre α vérifie :

$$0 < \alpha < 1.$$

En pratique, on choisit souvent une valeur proche de 0,8 : une valeur plus proche de 1 est plus fidèle à la structure du graphe, mais admet une complexité temporelle plus élevée (voir section 8.3).

3.2 Définition de la matrice de Google

On définit la matrice de Google G par :

$$G = \alpha P + \frac{1 - \alpha}{n} \mathbf{1},$$

où :

- P est la matrice de transition,
- n est le nombre de pages,
- $\mathbf{1}$ désigne la matrice dont toutes les entrées valent 1.

Ce qui revient, entrée par entrée, à écrire :

$$G_{ij} = \alpha P_{ij} + \frac{1 - \alpha}{n}.$$

3.3 Propriétés de la matrice G

La matrice G possède deux propriétés importantes :

- Toutes ses entrées sont strictement positives.
- La somme des coefficients de chaque colonne vaut 1.

G est donc une matrice stochastique stricte. C'est cette propriété qui garantit l'existence et l'unicité d'une distribution stationnaire, comme on le verra dans la section suivante.

3.4 Interprétation probabiliste

La matrice G décrit un processus de Markov modélisant la navigation d'un internaute aléatoire.

À chaque étape :

- avec probabilité α , il suit un lien hypertexte,
- avec probabilité $1 - \alpha$, il choisit une page uniformément au hasard.

Ce second comportement, la téléportation, assure que le surfeur peut atteindre n'importe quelle page du graphe, ce qui élimine les problèmes de réductibilité rencontrés avec P seule.

4 Modélisation par une chaîne de Markov

Pour modéliser la navigation d'un internaute sur le Web, on adopte une interprétation probabiliste du graphe en le décrivant comme une chaîne de Markov.

4.1 Vecteur de probabilité

On introduit le vecteur :

$$x_k = (x_k^{(1)}, \dots, x_k^{(n)}) \in \mathbb{R}^n$$

où $x_k^{(i)}$ représente la probabilité que le surfeur se trouve sur la page i à l'instant k . Par définition, x_k est un vecteur de probabilité et vérifie :

$$x_k^{(i)} \geq 0 \quad \text{et} \quad \sum_{i=1}^n x_k^{(i)} = 1.$$

À l'instant initial, on suppose que le surfeur a une probabilité uniforme de se trouver sur chacune des pages. On choisit donc :

$$x_0 = \left(\frac{1}{n}, \dots, \frac{1}{n} \right).$$

4.2 Évolution du système

La transition d'un instant k à l'instant $k + 1$ est gouvernée par la matrice de Google G , selon la relation :

$$x_{k+1} = Gx_k.$$

Cette écriture traduit le fait que la position du surfeur à l'instant $k + 1$ ne dépend que de sa position à l'instant k , c'est précisément la propriété de Markov.

4.3 Recherche d'un état stationnaire

On cherche un vecteur x^* vérifiant :

$$x^* = Gx^*.$$

Un tel vecteur est appelé *distribution stationnaire* de la chaîne de Markov. Il correspond à un régime d'équilibre où la distribution de probabilité ne se modifie plus d'une étape à l'autre.

4.4 Interprétation

La composante $x^{*(i)}$ a une signification concrète : elle représente la proportion du temps que le surfeur passe sur la page i lorsque le nombre d'étapes tend vers l'infini. Autrement dit, x^* capture la fréquence limite de visite de chaque page. Les pages associées aux plus grandes valeurs de $x^{*(i)}$ sont naturellement considérées comme les plus importantes. C'est sur cette idée que repose le classement PageRank.

5 Existence et unicité du PageRank

On cherche un vecteur de probabilité $x \in \mathbb{R}^n$ tel que :

$$Gx = x.$$

5.1 Existence et unicité d'un point fixe

Soit $P \in \mathcal{M}_n(\mathbb{R}_+)$ une matrice stochastique par colonnes, c'est-à-dire telle que $\mathbf{1}^T P = \mathbf{1}^T$, où $\mathbf{1} \in \mathbb{R}^n$ désigne le vecteur colonne dont toutes les entrées valent 1.

La matrice de Google est :

$$G := \alpha P + \frac{1 - \alpha}{n} \mathbf{1} \mathbf{1}^T,$$

où $0 < \alpha < 1$.

On a $G \in \mathcal{M}_n(\mathbb{R}_+)$ et on remarque que G est stochastique en colonne, en effet :

$$\begin{aligned} \mathbf{1}^T G &= \mathbf{1}^T \left(\alpha P + \frac{1 - \alpha}{n} \mathbf{1} \mathbf{1}^T \right) \\ &= \alpha \mathbf{1}^T P + \frac{1 - \alpha}{n} (\mathbf{1}^T \mathbf{1}) \mathbf{1}^T \\ &= \alpha \mathbf{1}^T + \frac{1 - \alpha}{n} \cdot n \cdot \mathbf{1}^T \\ &= \mathbf{1}^T. \end{aligned}$$

On note :

$$C = \left\{ x = (x_i)_{1 \leq i \leq n} \in \mathbb{R}_+^n : \sum_{i=1}^n x_i = 1 \right\}$$

l'ensemble des vecteurs de probabilité à valeurs positives.

Soient $X, Y \in C$. Puisque $\mathbf{1}^T X = \mathbf{1}^T Y = 1$, on a $\mathbf{1}^T (X - Y) = 0$, d'où :

$$GX - GY = \alpha P(X - Y) + \frac{1 - \alpha}{n} \underbrace{\mathbf{1} \mathbf{1}^T (X - Y)}_{=0} = \alpha P(X - Y).$$

Comme $\alpha > 0$, en norme 1, cela donne :

$$\|GX - GY\|_1 = \alpha \|P(X - Y)\|_1$$

Par inégalité triangulaire sur les normes matricielles, on obtient :

$$\|GX - GY\|_1 \leq \alpha \|P\| \cdot \|X - Y\|_1,$$

où $\|P\| = \max_{x \in C} \|Px\|_1$.

Pour tout $x \in C$, on calcule :

$$\|Px\|_1 = \sum_{i=1}^n [Px]_i = \sum_{i=1}^n \sum_{k=1}^n P_{ik} x_k = \sum_{k=1}^n x_k \sum_{i=1}^n P_{ik}.$$

Or P est stochastique par colonnes, donc $\sum_{i=1}^n P_{ik} = 1$ pour tout $k \in \{1, \dots, n\}$.

De plus, $x \in C$ implique $\sum_{k=1}^n x_k = 1$, d'où $\|Px\|_1 = 1$.

Ainsi $\|P\| = 1$, et on obtient :

$$\forall X, Y \in C, \quad \|GX - GY\|_1 \leq \alpha \|X - Y\|_1.$$

G est donc α -contractante sur C . C est une partie fermée de $\mathbb{R}^n$, donc un espace métrique complet pour la norme $\|\cdot\|_1$. Le théorème du point fixe de Banach-Picard assure alors l'existence et l'unicité d'un point fixe x^* .

5.2 Conséquence pour le PageRank

On est donc assuré de l'existence d'un unique vecteur x^* vérifiant :

$$Gx^* = x^*$$

avec :

$$x_i^* \geq 0 \quad \text{et} \quad \sum_{i=1}^n x_i^* = 1.$$

Ce vecteur correspond à la distribution stationnaire du surfeur aléatoire. Autrement dit, c'est la fréquence limite à laquelle chaque page est visitée. Ses composantes constituent les scores de PageRank.

De plus, le théorème de Banach-Picard nous dit qu'en considérant, pour un x_0 arbitraire, la relation $x_{n+1} = Gx_n$, la suite $(x_n)_{n \in \mathbb{N}}$ converge géométriquement vers x^* .

En effet, on peut montrer par récurrence que :

$$\forall n \in \mathbb{N}, \quad \|G^n x_0 - x^*\| \leq \alpha^n \|x_0 - x^*\|$$

Cette convergence permettra, informatiquement, de mettre en place un algorithme pour trouver cet état stationnaire. On choisira arbitrairement un seuil ε tel qu'il existera un rang m pour lequel :

$$\|G^m x_0 - x^*\| \leq \varepsilon,$$

afin de garantir que l'algorithme se termine.

6 Implémentation en Python

Pour illustrer concrètement le modèle présenté, on a implémenté l'algorithme PageRank en Python. L'implémentation s'appuie principalement sur **numpy** pour les calculs matriciels et **networkx** pour la manipulation et la visualisation des graphes.

Dans un premier temps, on travaille avec des matrices pleines, ce qui simplifie l'implémentation et permet de mieux suivre chaque étape de l'algorithme. La question des matrices creuses, plus réalistes au regard de la structure réelle du Web, sera abordée dans un second temps.

6.1 Structure du programme

Le programme est organisé en plusieurs fonctions Python correspondant aux différentes étapes du calcul du PageRank :

- `graph_alea(n)` : génère une matrice d'adjacence aléatoire.
- `calcul_P(A,n)` : construit la matrice de transition.
- `calcul_G(A,n)` : construit la matrice de Google.
- `surfer_alea(n)` : génère un vecteur de probabilité initial.
- `pagerank(G,n)` : calcule le vecteur PageRank par itération.

Ce découpage permet de suivre clairement les différentes étapes du calcul et de séparer l'analyse de la complexité de chaque partie.

6.2 Génération du graphe

Le Web est modélisé par une matrice d'adjacence $A \in \{0,1\}^{n \times n}$, générée aléatoirement dans notre programme pour simuler un réseau de pages web. Chaque coefficient a_{ij} vaut 1 si la page j possède un lien vers la page i , et 0 sinon. Cette matrice sert de point de départ pour construire le graphe orienté associé et calculer les degrés sortants, nécessaires à l'étape suivante.

La génération de la matrice nécessite de remplir n^2 coefficients, ce qui donne une complexité en $O(n^2)$.

6.3 Construction de la matrice de transition

À partir de A , on construit la matrice de transition P définie par :

$$P_{ij} = \begin{cases} \frac{1}{\deg(j)} & \text{si } j \rightarrow i \\ 0 & \text{sinon} \end{cases}$$

où $\deg(j)$ désigne le degré sortant de la page j . En pratique, ce calcul revient à parcourir les colonnes de A et à normaliser chacune par la somme de ses entrées non nulles :

```
def calcul_P(A,n):  
  
    P = np.zeros((n,n))  
  
    for j in range(n):  
  
        deg = np.sum(A[:,j])  
  
        if deg != 0:  
            for i in range(n):  
                if A[i,j] == 1:  
                    P[i,j] = 1/deg  
  
    return P
```

Complexité : La fonction parcourt l'ensemble des colonnes de la matrice et examine leurs coefficients. Deux boucles de taille n , le coût total est donc en $O(n^2)$.

6.4 Construction de la matrice de Google

La matrice de Google est ensuite obtenue à partir de P via la formule :

$$G = \alpha P + \frac{1 - \alpha}{n} \mathbf{1}.$$

Le paramètre α est donné en argument de nos fonctions pour permettre plusieurs manipulations par la suite (voir section 8.3).

On l'implémente de cette manière :

```
def calcul_G(A,n, alpha):  
  
    P = calcul_P(A,n)  
    U = np.ones((n,n))  
  
    G = alpha*P + (1-alpha)/n * U  
  
    return G
```

Complexité : La construction de la matrice nécessite, la construction de P en $O(n^2)$, celle de U en $O(n^2)$ aussi, ainsi que des opérations sur les n^2 coefficients de la matrice. Ce qui donne une complexité en $O(n^2)$ également.

6.5 Calcul du PageRank

On construit le vecteur initial x_0 en créant un tableau de taille n rempli de 1, que l'on divise ensuite par n pour obtenir le vecteur uniforme $x_0 = (1/n, \dots, 1/n)$. La complexité de `surfer_alea(n)` est donc en $O(n)$.

Le vecteur PageRank est ensuite calculé par la méthode d'itération suivante :

$$x_{k+1} = Gx_k.$$

On répète le produit matrice-vecteur jusqu'à convergence. Le critère d'arrêt retenu, pour ε choisi arbitrairement, est :

$$\max_i |x_{k+1,i} - x_{k,i}| < \varepsilon.$$

On mesure ainsi l'écart maximal entre deux itérations consécutives composante par composante. Lorsque cet écart devient inférieur à ε , on considère que la suite a convergé et on renvoie le vecteur courant comme approximation de x^* .

La fonction renvoie également la liste des erreurs successives, ce qui permet d'analyser la vitesse de convergence de l'algorithme (voir section 7.2).

Son implémentation est la suivante :

```
def pagerank(A,n,x,alpha) :  
    G = calcul_G(A,n,alpha)  
  
    erreurs = []  
  
    diff = 1  
  
    while diff > epsilon :  
  
        xnext = np.dot(G,x)
```



```

diff = np.max(np.abs(x - xnext))

erreurs.append(diff)

x = xnext

return xnext, erreurs

```

Complexité : Chaque itération correspond à un produit matrice vecteur, dont le coût est en $O(n^2)$, et la boucle est itérée k fois, k étant le nombre d'itérations nécessaires pour atteindre la convergence. On verra par la suite que k dépend fortement de α (voir section 8.3). En pratique, le nombre d'itérations nécessaires pour atteindre la convergence reste faible devant n , donc la complexité de l'algorithme reste en $O(n^2)$.

7 Premiers résultats

Afin d'étudier le comportement de l'algorithme PageRank, nous avons réalisé plusieurs expériences numériques sur des graphes générés aléatoirement.

Dans un premier temps, nous utilisons des matrices pleines afin de simplifier l'implémentation. L'utilisation de matrices creuses, plus adaptées à la structure réelle du Web, sera discutée dans la section suivante (8.1).

Dans l'expérience suivante, pour mieux visualiser, nous choisissons arbitrairement :

$$n = 10 ; \alpha = 0,85 ; \varepsilon = 10^{-16}$$

7.1 Graphe généré et scores PageRank

On génère une matrice d'adjacence aléatoire $A \in \{0,1\}^{n \times n}$ et on calcule le vecteur PageRank associé. Les figures 1 et 2 ci-dessous présentent le graphe orienté obtenu ainsi que la visualisation des scores : la taille de chaque sommet est proportionnelle à son score PageRank, ce qui permet d'identifier visuellement les pages les plus importantes.

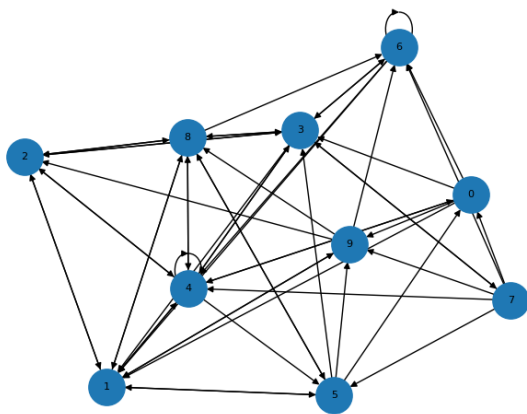


FIGURE 1 – Graphe orienté généré aléatoirement

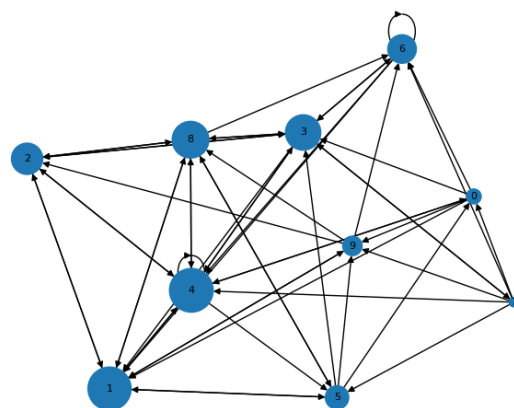


FIGURE 2 – Scores PageRank associés

On observe que certaines pages se détachent nettement des autres, ce qui correspond aux sommets recevant davantage de liens entrants.

7.2 Convergence de l'algorithme

L'algorithme PageRank repose sur l'itération

$$x_{k+1} = Gx_k.$$

Nous observons que la suite (x_k) converge rapidement vers la distribution stationnaire.

La figure 3 représente l'évolution de l'erreur entre deux itérations successives, lors de l'expérience précédente.

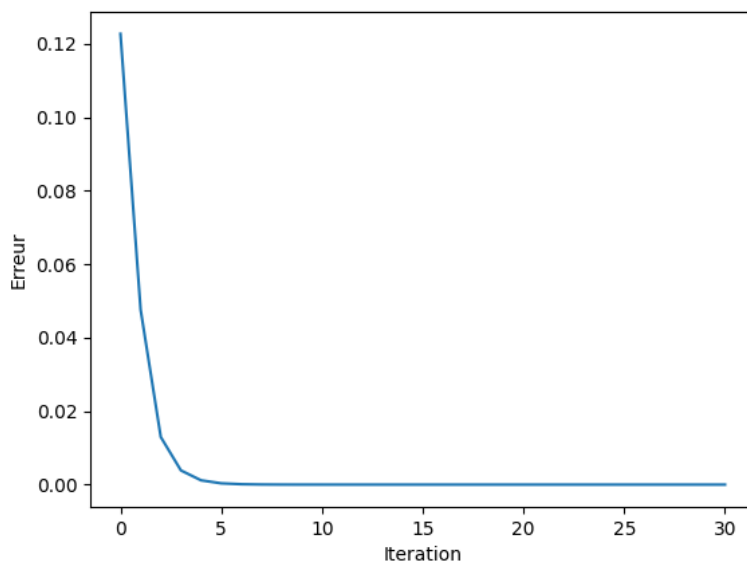


FIGURE 3 – Convergence de l'algorithme PageRank

8 Réalisme et Optimisations

8.1 Utilisation de matrices creuses

Dans l'implémentation précédente, les matrices sont stockées sous forme pleine de taille $n \times n$, ce qui représente un coût mémoire de l'ordre de $O(n^2)$. Pour de grandes valeurs de n , cela devient vite problématique. D'autant plus que dans un réseau web réel, chaque page ne pointe que vers un nombre limité d'autres pages : la matrice d'adjacence est donc très majoritairement composée de zéros, ce qui rend la représentation pleine inefficace et non réaliste.

On passe donc à une représentation creuse au format CSC (*Compressed Sparse Column*). Ce format stocke uniquement les entrées non nulles de la matrice, organisées en trois tableaux :

- les valeurs non nulles,
- les indices de ligne correspondants,
- les indices de début de chaque colonne dans le tableau des valeurs.

Ce format est particulièrement adapté au produit matrice-vecteur colonne par colonne, qui est précisément l'opération centrale de l'itération PageRank. La complexité par itération tombe alors à $O(m)$, où m est le nombre total d'arêtes du graphe. Le produit matrice-vecteur de l'itération PageRank ne dépend alors plus de n^2 , mais uniquement du nombre total d'arêtes m du graphe, ce qui fait tomber la complexité par itération à $O(m)$.

Choix du nombre moyen de liens par page : Pour générer des graphes creux réalistes, on choisit un nombre moyen de liens par page égal à $\lfloor \log_{10}(n) \rfloor$. Ce choix est motivé par deux observations :

- En pratique, le nombre de liens sortants d’une page web est faible et croît très lentement avec la taille du Web. $\log_{10}$ est une fonction à croissance très lente : pour $n = 1\,000$ pages, on obtient 3 liens par page en moyenne ; pour $n = 1\,000\,000$, seulement 6. Cela reflète bien le fait qu’une page ne pointe que vers une poignée d’autres, indépendamment de la taille globale du réseau.
- Ce choix garantit que $m = n \lfloor \log_{10}(n) \rfloor \ll n^2$ dès que n est grand, ce qui assure un gain effectif par rapport à la représentation dense.

Le nombre de liens de chaque page est tiré selon une loi de Poisson de paramètre $\lfloor \log_{10}(n) \rfloor$. Ce choix est naturel pour modéliser un comptage d’événements rares et indépendants : ici, chaque lien sortant peut être vu comme un événement indépendant, et la loi de Poisson garantit que le nombre moyen de liens par page vaut bien λ , tout en permettant une variabilité réaliste autour de cette valeur (certaines pages auront plus de liens, d’autres moins).

Comparaison expérimentale : Pour quantifier le gain en temps de calcul, on a mesuré le temps d’exécution de l’algorithme sur deux représentations d’un même graphe. Pour chaque taille $n = 200, 400, \dots, 2\,000$, on a généré 1 000 graphes aléatoires distincts, et pour chacun d’eux, on a exécuté l’algorithme PageRank successivement sur la représentation dense et sur la représentation creuse du même graphe. Le temps moyen sur ces 1 000 graphes est représenté sur la figure 4.

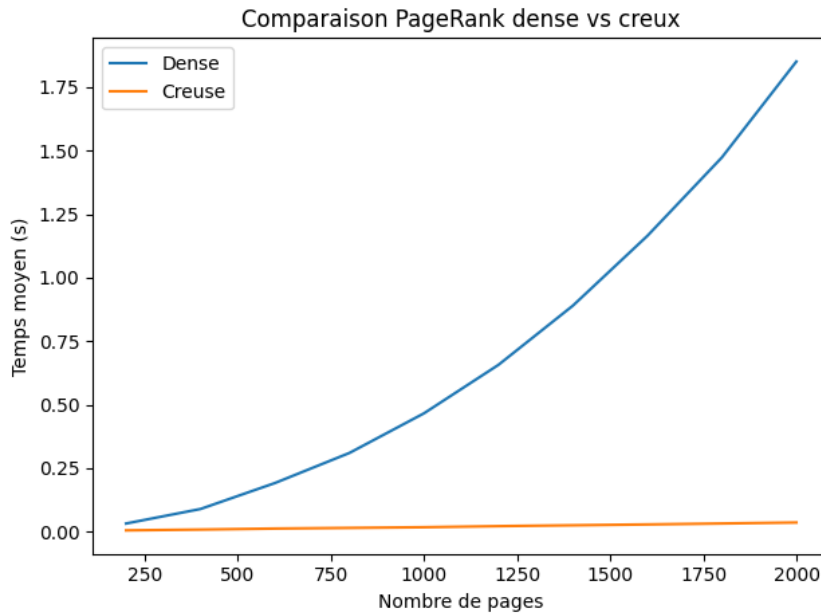


FIGURE 4 – Comparaison du temps de calcul algo entre matrices denses et creuses

On observe que le temps d’exécution pour la représentation dense croît de façon quasi quadratique avec n , ce qui est cohérent avec la complexité théorique en $O(n^2)$ par itération. Malgré le faible nombre de lien, l’algorithme parcourt toujours toute la matrice.

À l’inverse, le temps pour la représentation creuse reste quasi nul sur toute la plage de valeurs testées : le faible nombre de liens par page ($\lfloor \log_{10}(n) \rfloor$) maintient $m \ll n^2$, et le gain apporté par le format CSC est donc particulièrement marqué dès que n devient grand.

8.2 Exemple sur un graphe creux

On illustre ici le comportement de l'algorithme sur un graphe creux de taille $n = 65$, généré aléatoirement. Les figures 5 et 6 présentent la structure brute du graphe ainsi que les scores PageRank associés, pour $\alpha = 0,85$.

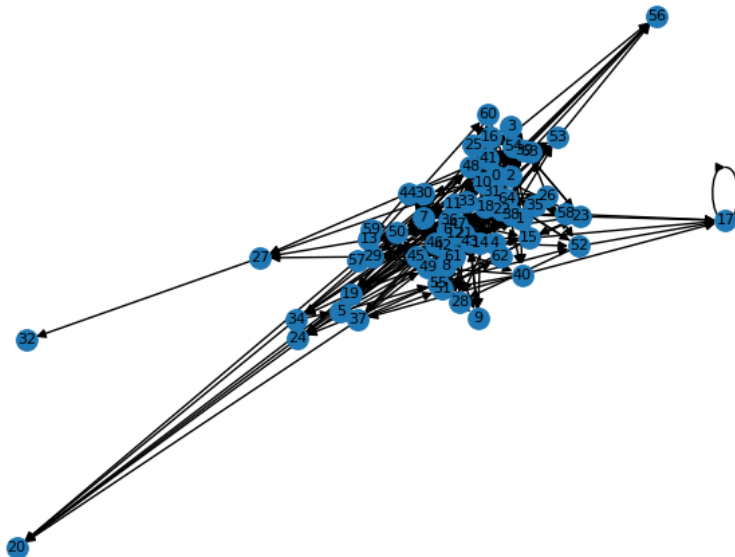


FIGURE 5 – Structure du graphe ($n = 65$)

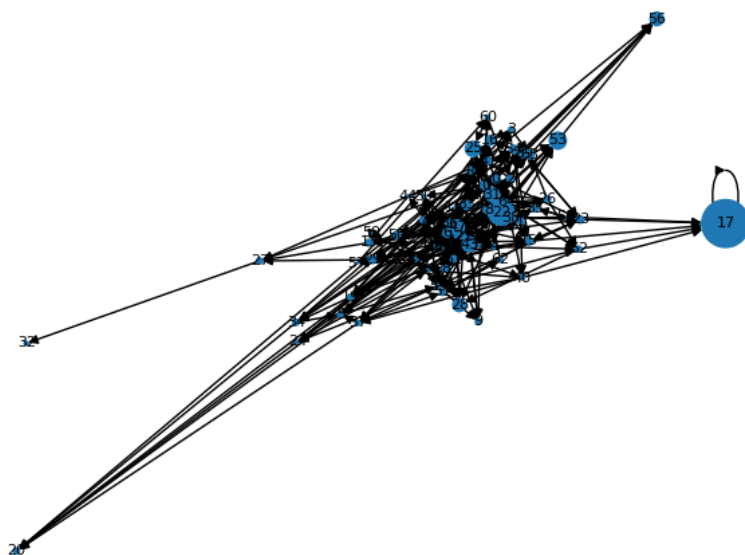


FIGURE 6 – Scores PageRank associés ($\alpha = 0,85$)

La représentation creuse permet de ne faire apparaître qu'un faible nombre de liens, ce qui évite la saturation visuelle produite par un graphe dense de même taille. La structure générale du graphe reste assez visible.

On observe que certaines pages se détachent nettement des autres. Le nœud le plus grand concentre un score PageRank bien supérieur à la moyenne : il reçoit un grand nombre de liens

entrants, ce qui lui confère une importance élevée aux yeux de l'algorithme. À l'opposé, des nœuds apparaissent très petits, presque invisibles : peu référencés et disposant de peu de liens sortants, ils ne bénéficient presque que du facteur de téléportation. Ces nœuds ne sont pas complètement isolés : ils entretiennent quelques connexions, mais celles-ci sont insuffisantes pour leur faire gagner un score significatif.

8.3 Influence du paramètre α

Vitesse de Convergence : Le paramètre α contrôle la probabilité que le surfeur suive un lien hypertexte plutôt que de se téléporter vers une page aléatoire. Il joue un rôle sur deux aspects distincts : le comportement du modèle et la vitesse de convergence de l'algorithme.

La figure 7 montre l'évolution de l'erreur $\|x_{k+1} - x_k\|_\infty$ au fil des itérations, pour une matrice creuse, différentes valeurs de α et $n = 200$.

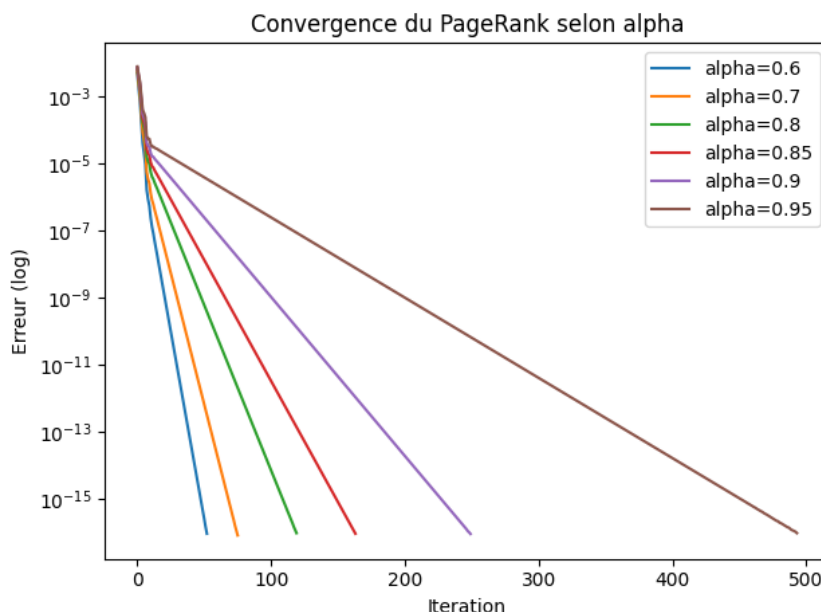


FIGURE 7 – Influence du paramètre α sur la convergence

L'axe vertical est représenté avec une échelle logarithmique. Ce choix permet de visualiser plus clairement la décroissance de l'erreur, qui devient très rapide après quelques itérations. Avec une échelle linéaire, les petites valeurs de l'erreur seraient écrasées près de zéro et les différences de vitesse de convergence entre les différentes valeurs de α seraient beaucoup moins visibles et impossibles à comparer.

Sans grande surprise, les valeurs faibles de α convergent plus vite : la téléportation vers une page aléatoire domine, ce qui homogénéise rapidement la distribution.

À l'inverse, les valeurs proches de 1 accordent davantage de poids à la structure du graphe, au prix d'une convergence plus lente.

En pratique, $\alpha = 0,85$ constitue un compromis raisonnable : la structure du graphe reste prépondérante dans le calcul du PageRank, tout en conservant une vitesse de convergence satisfaisante. C'est d'ailleurs la valeur historiquement utilisée par Google.

α et PageRank : On s'intéresse maintenant à l'effet de α sur la distribution des scores PageRank elle-même. Pour cela, on fixe un même graphe creux de taille $n = 20$, puis on calcule le PageRank pour plusieurs valeurs de α .

Les graphes obtenus sont ensuite représentés, l'objectif étant de comparer l'importance relative accordée à la structure du graphe et à la téléportation aléatoire selon les différentes valeurs de α .

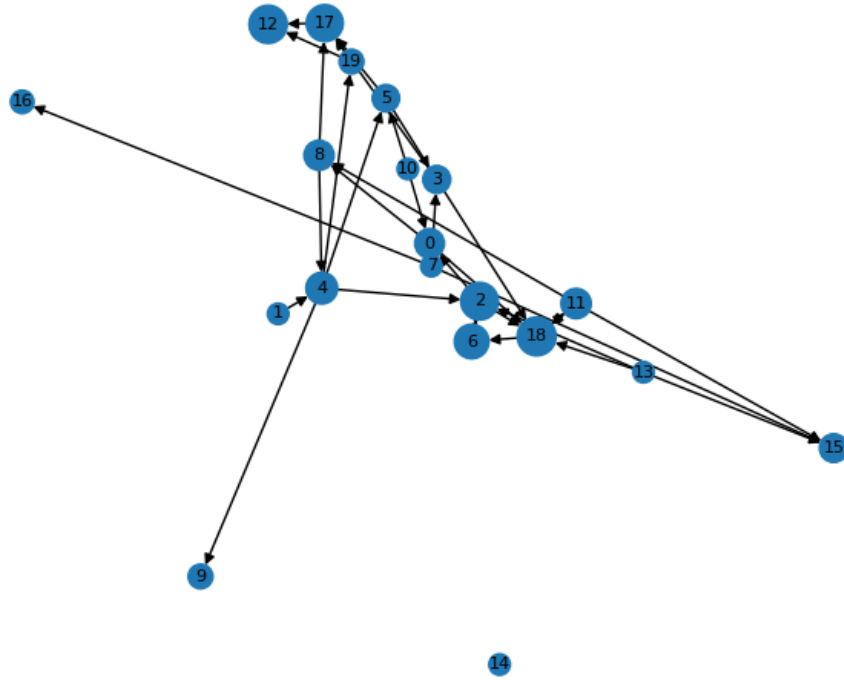


FIGURE 8 – Distribution du PageRank pour $\alpha = 0,6$

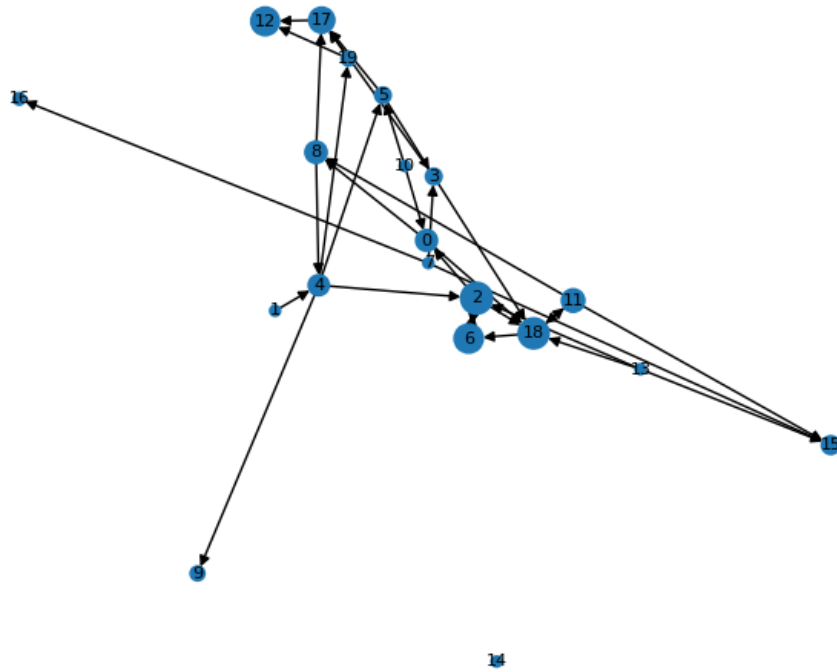


FIGURE 9 – Distribution du PageRank pour $\alpha = 0,9$

Pour $\alpha = 0,6$ (8), les scores sont relativement proches les uns des autres, ce qui reflète un comportement assez uniforme. La téléportation efface en partie les différences de structure. Pour $\alpha = 0,9$ (9) en revanche, certaines pages se détachent nettement : la topologie du graphe reprend le dessus, et les sommets bien connectés voient leur score augmenter de façon marquée. Le paramètre α agit donc bien comme un curseur entre exploration aléatoire et fidélité à la structure des liens.

9 Conclusion

Le classement de pages web à grande échelle pourrait sembler relever avant tout de la puissance de calcul brute. Face à un graphe de plusieurs milliards de nœuds, deux stratégies s'offrent en effet : augmenter les ressources disponibles, ou trouver une modélisation qui rende le problème structurellement plus simple. PageRank emprunte résolument la seconde voie.

Plutôt que d'analyser localement chaque lien, l'algorithme exploite une représentation probabiliste du graphe via les chaînes de Markov. Le classement émerge alors d'une propriété globale : la distribution stationnaire, capturée par une équation de point fixe aussi compacte qu'efficace, dont l'existence et l'unicité sont garanties par le théorème de Banach-Picard. Ce changement de point de vue est, rétrospectivement, ce qui rend l'approche aussi élégante que puissante.

Cela dit, la pertinence mathématique ne suffit pas. Comme on l'a vu avec la comparaison entre matrices denses et matrices creuses au format CSC, le passage à une représentation adaptée à la structure réelle du graphe réduit drastiquement le coût de chaque itération, de $O(n^2)$ à $O(m)$, ce qui change tout dès que n dépasse quelques milliers de pages. Le paramètre α illustre de son côté un compromis inhérent au modèle : donner trop de poids à la téléportation efface les différences de structure ; trop peu ralentit la convergence. La valeur $\alpha = 0,85$ historiquement retenue par Google reflète un équilibre empiriquement validé entre ces deux extrêmes.

Le modèle a toutefois ses limites. Il repose sur une vision assez schématique du comportement des utilisateurs, sans tenir compte du contenu des pages, des requêtes formulées, ni des préférences individuelles. PageRank constitue une base mathématique solide, mais les moteurs de recherche modernes l'ont depuis longtemps enrichi de nombreuses autres couches de traitement.

Ce projet aura au moins montré une chose : des outils relativement accessibles : graphes orientés, matrices stochastiques, itération de puissance ; suffisent à traiter un problème d'une complexité réelle, à condition de les combiner avec soin.

Cependant, même ramenée à $O(m)$ par itération, la mise à l'échelle réelle du Web pose des défis considérables. Google indexe aujourd'hui plusieurs centaines de milliards de pages ; avec un degré moyen de quelques dizaines de liens par page, m se chiffre en milliers de milliards d'opérations par itération, répétées plusieurs fois par jour pour tenir compte des mises à jour du Web. Le coût énergétique de ces calculs est loin d'être négligeable, et ce constat résonne avec un débat bien plus contemporain : l'essor des grands modèles d'intelligence artificielle a placé la question du coût énergétique du calcul au cœur de l'actualité. Optimiser les algorithmes n'est donc pas seulement une question d'efficacité, c'est également une question de sobriété.

Annexe

```
"""
Implementation et analyse de l'algorithme PageRank.

Fonctionnalites :
    Generation de graphes aleatoires (dense et creux)
    Calcul du PageRank
    Visualisation du graphe avec tailles de noeuds proportionnelles au
    PageRank
    Etude de la convergence en fonction du parametre alpha
    Comparaison des performances dense vs creux
"""

import time

import matplotlib.pyplot as plt
import networkx as nx
import numpy as np
from scipy.sparse import csc_matrix, csr_matrix

epsilon = 1e-16
ALPHA_DEFAULT = 0.85

# Generation de graphes aleatoires
def graph_alea(n, methode="dense"):
    """
    Genere une matrice d'adjacence aleatoire.

    methode="dense" : matrice n*n binaire uniforme
                     Complexite :  $O(n^2)$ 

    methode="creux" : chaque noeud a en moyenne  $\log_{10}(n)$  liens sortants.
                     Retourne une csc_matrix de forme (n, n).
                     Complexite :  $O(n \cdot \log n)$ 

    """
    if methode == "dense":
        A = np.random.randint(2, size=(n, n))

    else :
        d = int(np.log10(n)) # degre moyen cible
        p = d / n           # probabilite binomiale associee
        indptr = [0]
        indices = []
        data = []

        for j in range(n):
            k = np.random.binomial(n, p) # nombre de liens sortants de j
            if k > 0:
                col = np.random.choice(n, size=k, replace=False)
                col.sort()
                indices.extend(col)
                data.extend([1] * k)
            indptr.append(len(indices))
        A = csc_matrix((data, indices, indptr), shape=(n, n))
```



```

    return A

# Conversion de la matrice en graphe oriente NetworkX
def mat_to_graph(A, n, methode="dense"):
    """
    Convertit une matrice d'adjacence A en graphe oriente NetworkX.
    A[i,j] = 1 signifie qu'il y a un lien de la page j vers la page i.

    Pour methode="creux", A est convertie en dense avant parcours
    (usage reserve a la visualisation de petits graphes).

    Complexite :  $O(n^2)$ 
    """
    G = nx.DiGraph()
    G.add_nodes_from(range(n))

    if methode == "creux":
        A = A.toarray()

    for i in range(n):
        for j in range(n):
            if A[i, j] == 1:
                G.add_edge(j, i)

    return G

# Vecteur initial du surfer
def surfer_alea(n):
    """
    Genere un vecteur de probabilite uniforme
    representant la position initiale du surfer.

    Complexite :
     $O(n)$ 
    """

    x = np.ones(n) / n

    return x

# Affichage du graphe
def affiche_graph(A, n, f=None, methode="dense") :
    """
    Affiche le graphe.

    Complexite : Complexite de la fonction draw : couteuse pour des
    grandes valeurs de n.
    """

    Ggraph = mat_to_graph(A, n, methode)
    pos = nx.spring_layout(Ggraph, seed=42)
    # Positionnement des noeuds pour une meilleure visualisation.

    plt.figure("Graphe :")
    nx.draw(Ggraph,
            pos,
            with_labels=True,
            font_size=8,
            node_size=8000/n)

```

```

#ligne suivante pour enregistrer le graphe et l'inserer au rapport.
if f :
    plt.savefig(f)

plt.show()

return

# Matrice de transition P
def calcul_P(A, n, methode="dense"):
    """
    Calcule la matrice de transition P du graphe.
    P[i,j] = probabilite de passer de j vers i

    methode="dense" : A est une matrice n*n
                     Complexite :  $O(n^2)$ 

    methode="sparse" : A est une csr_matrix. (matrice creuse au format
                     colonne)
                     Complexite :  $O(\text{nnz}(A)) = O(n \cdot \log n)$ , avec  $\text{nnz}(A)$ 
                     le nombre d'entrees non nulles de A.
    """
    if methode == "dense":

        P = np.zeros((n, n))

        for j in range(n):

            #Calcul du degre sortant de j
            deg = np.sum(A[:, j])

            #remplissage de la colonne j de P
            if deg != 0:

                for i in range(n):

                    if A[i, j] == 1:
                        P[i, j] = 1.0 / deg

    else :
        rows = []
        cols = []
        data = []

        #parcours colonne par colonne
        for j in range(n):

            debut = A.indptr[j]
            fin = A.indptr[j+1]

            #degre de la colonne j :
            deg = fin - debut

            if deg > 0:

                for i in A.indices[debut:fin] :
                    rows.append(i)
                    cols.append(j)
                    data.append(1.0 / deg)

```

```

        P = csr_matrix((data, (rows, cols)), shape=(n, n))
    return P

# Algorithme de PageRank
def pagerank(A, n, x, methode="dense", alpha=0.85):
    """
    Calcule le vecteur PageRank par iteration :

         $x_{k+1} = \alpha * P * x_k + (1-\alpha)/n * 1$ 

    jusqu'a convergence.

    Critere d'arret :
        max  $|x_{k+1} - x_k| < \epsilon$  : toutes les composantes du vecteur
        de probabilite sont proches de leur valeur limite.

    Methode "dense" : on utilise np.dot donc la complexite est en
     $O(n^2 * k)$  avec k le nombre d'iterations necessaires pour atteindre
    la convergence.
    Methode "creux" : on utilise P.dot(x) donc la complexite est en
     $O(\text{nnz}(P) * k) = O(n * \log n * k)$ .
    """

    teleport = (1.0 - alpha) / n
    erreurs = []

    P = calcul_P(A, n, methode)

    diff = 1

    while diff > epsilon :
        if methode == "dense":
            xnext = alpha * np.dot(P, x) + teleport
        else :
            xnext = alpha * P.dot(x) + teleport

        diff = np.max(np.abs(x - xnext))
        erreurs.append(diff)
        x = xnext

    return xnext, erreurs

# Visualisation du PageRank
def draw_pagerank(A, n, alpha, f=None, methode="dense"):
    """
    Affiche le graphe obtenu avec des tailles de noeuds proportionnelles
    au PageRank.
    """

    Ggraph = mat_to_graph(A, n, methode)
    x0 = surfer_alea(n)

    scores, _ = pagerank(A, n, x0, methode, alpha)
    sizes = 50000*scores / n
    pos = nx.spring_layout(Ggraph, seed=42)

    plt.figure("Graphe avec PageRank :")

```

```

nx.draw(Ggraph,
        pos,
        with_labels=True,
        font_size=8,
        node_size=sizes)

if f:
    plt.savefig(f)

plt.show()

return

# etude de la convergence selon alpha
def convergence_alpha(A, n, x, alphas=None, methode="dense"):
    """
    Etudie la convergence de l'algorithme de PageRank en fonction de
    alpha.
    On remarque que plus alpha est proche de 1, plus la convergence est
    lente, mais plus alpha est proche de 1, plus le PageRank depend du
    graphe et moins du hasard.

    On cherche un bon candidat pour alpha, qui soit a la fois rapide a
    converger et qui reflete bien la structure du graphe.

    methode : "dense" ou "creux"
    """

    if alphas is None :
        alphas = [0.6,0.7,0.8,0.85,0.9,0.95]

    plt.figure()

    for a in alphas:

        _, erreurs = pagerank(A, n, x.copy(), methode, a)
        plt.semilogy(erreurs,label=f"alpha={a}")

    plt.xlabel("Iteration")
    plt.ylabel("Erreur (log)")
    plt.title("Convergence du PageRank selon alpha")
    plt.legend()
    plt.tight_layout()
    plt.savefig("convergence_alpha.png")
    plt.show()

    return

# Comparaison des performances : dense vs creux
def comparaison(sizes, essais=100, alpha = ALPHA_DEFAULT):
    """
    Mesure et compare les temps d'execution du PageRank dense et creux
    pour differentes tailles de graphe.

    Parametres :
        sizes : liste de valeurs de n a tester
        essais : nombre de repetitions pour moyenner les temps
    """

```

Les deux methodes partent du meme graphe creux converti en dense,
et du meme vecteur de surfer initial.
Affiche une courbe temps moyen en fonction de la taille du graphe.
"""

```
temps_dense = []
temps_creux = []

for n in sizes:
    print(f"Comparaison n={n}...")
    td, ts = 0.0, 0.0
    x = surfer_alea(n)

    for _ in range(essais):

        A_creux = graph_alea(n, "creux")
        A_dense = A_creux.toarray()

        # Dense
        start = time.time()

        pagerank(A_dense, n, x.copy(), "dense", alpha)
        td += time.time() - start

        # Creux
        start = time.time()
        pagerank(A_creux, n, x.copy(), "creux", alpha)
        ts += time.time() - start

    temps_dense.append(td / essais)
    temps_creux.append(ts / essais)

plt.figure()
plt.plot(sizes, temps_dense, label="Dense")
plt.plot(sizes, temps_creux, label="Creuse")
plt.xlabel("Nombre de pages")
plt.ylabel("Temps moyen (s)")
plt.title("Comparaison PageRank dense vs creux")
plt.legend()
plt.tight_layout()
plt.savefig("comparaison_dense_vs_creux.png")
plt.show()
return
```

Programme principal

```
def main():

    # Parametres
    n = int(input("Nombre de pages : "))
    assert n > 0, "Le nombre de pages doit etre positif."

    alpha_values = [0.85]
    methode = "creux"
    x0 = surfer_alea(n)

    # Generation
    A = graph_alea(n, methode)

    filename1 = f"graphe_n{n}_{methode}.png"
```

```
affiche_graph(A, n, filename1, methode)

for a in alpha_values:
    filename2 = f"pagerank_alpha_{int(a*100)}_{methode}.png"
    draw_pagerank(A, n, a, filename2, methode)

# Etude de convergence selon alpha

convergence_alpha(A, n, x0.copy(), methode=methode)

# Comparaison dense vs creux
comparaison(sizes=[200, 400, 600, 800, 1000, 1200, 1400, 1600,
                  1800, 2000], essais=1000)

return

if __name__ == "__main__":
    main()
```